

SQL Joins

Combining Data From Multiple Tables

Lab Manual — Lab #2

Course	Database Systems
Topic	INNER, LEFT, RIGHT, FULL OUTER, SELF, and multi-table joins
Duration	One lab session (approx. 3 hours)
Tools	MySQL 8.x, MySQL Workbench / Command Line Client
Prerequisites	Lab #1 (SQL Filters), foreign keys, normalization
Builds On	Lab #1 — students must be comfortable with WHERE filters

1. Learning Objectives

By the end of this lab, students will be able to:

- 1 Explain why joins are needed once data has been normalized into multiple tables.
- 2 Describe the Cartesian product and why a join condition is essential.
- 3 Write **INNER JOIN** queries to combine matching rows from two or more tables.
- 4 Write **LEFT JOIN** and **RIGHT JOIN** queries to preserve unmatched rows.
- 5 Identify when to use INNER vs LEFT/RIGHT joins for a given business question.
- 6 Emulate **FULL OUTER JOIN** in MySQL using LEFT and RIGHT joins combined with UNION.
- 7 Use **SELF JOIN** to relate a table to itself (e.g. employees and their managers).
- 8 Chain three or more tables in a single query.
- 9 Use the **anti-join pattern** (LEFT JOIN + IS NULL) to find missing relationships.
- 10 Combine joins with WHERE filters from Lab #1 to answer realistic business questions.

Outcome

Each student will independently write multi-table SQL queries on a given schema, producing a single .sql file with all answers, ready to be pushed to their GitHub repository.

2. Session Plan (3 Hours)

Time	Activity	Mode
0:00 – 0:10	Recap of Lab #1 + lab schema setup (run provided script)	Setup
0:10 – 0:30	Why joins? Cartesian product, join condition, INNER JOIN	Lecture + Demo
0:30 – 0:55	LEFT JOIN, RIGHT JOIN — when each is needed	Lecture + Demo
0:55 – 1:25	Lab Tasks Part A — INNER, LEFT, RIGHT joins on two tables	Hands-on
1:25 – 1:35	Short break	—
1:35 – 1:55	FULL OUTER JOIN (via UNION) and SELF JOIN	Lecture + Demo
1:55 – 2:15	Multi-table joins (3+ tables) and the anti-join pattern	Lecture + Demo
2:15 – 2:55	Lab Tasks Part B — Self joins, multi-table joins, joins + filters	Hands-on
2:55 – 3:00	Wrap-up, common mistakes, deliverables	Discussion

3. The Lab Schema — Company Database

This lab uses a **Company** database with four related tables: *Department*, *Employee*, *Project*, and *Assignment* (a junction table linking employees to projects). The schema deliberately contains gaps — empty departments, unassigned employees, projects without departments — so that LEFT/RIGHT joins produce meaningful results.

3.1 Tables and Relationships

Table	Columns	Keys
Department	DeptID, DeptName, Location, Budget	PK: DeptID
Employee	EmpID, EmpName, Gender, Salary, HireDate, City, ManagerID, DeptID	PK: EmpID; FK: DeptID → Department; FK: ManagerID → Employee
Project	ProjectID, ProjectName, StartDate, EndDate, DeptID	PK: ProjectID; FK: DeptID → Department
Assignment	EmpID, ProjectID, HoursPerWeek	PK: (EmpID, ProjectID); FK: EmpID; FK: ProjectID

3.2 Setup Script — Run this First

```

CREATE DATABASE IF NOT EXISTS joins_lab;
USE joins_lab;

DROP TABLE IF EXISTS Assignment, Project, Employee, Department;

CREATE TABLE Department (
    DeptID    INT PRIMARY KEY,
    DeptName  VARCHAR(40) NOT NULL,
    Location  VARCHAR(30),
    Budget    DECIMAL(12,2)
);

CREATE TABLE Employee (
    EmpID     INT PRIMARY KEY,
    EmpName   VARCHAR(50) NOT NULL,
    Gender    CHAR(1),
    Salary    DECIMAL(10,2),
    HireDate  DATE,
    City      VARCHAR(30),
    ManagerID INT,
    DeptID    INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID),
    FOREIGN KEY (ManagerID) REFERENCES Employee(EmpID)
);

CREATE TABLE Project (
    ProjectID INT PRIMARY KEY,
    ProjectName VARCHAR(50) NOT NULL,
    StartDate  DATE,
    EndDate    DATE,
    DeptID     INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

CREATE TABLE Assignment (
    EmpID     INT,

```

```

    ProjectID      INT,
    HoursPerWeek INT,
    PRIMARY KEY (EmpID, ProjectID),
    FOREIGN KEY (EmpID)      REFERENCES Employee(EmpID),
    FOREIGN KEY (ProjectID) REFERENCES Project(ProjectID)
);

```

3.3 Sample Data Inserts

```

-- Departments
INSERT INTO Department VALUES
(10, 'Engineering', 'Lahore', 5000000),
(20, 'Marketing', 'Karachi', 2000000),
(30, 'Finance', 'Islamabad', 3000000),
(40, 'Research', 'Lahore', 4000000),
(50, 'Sales', 'Karachi', NULL); -- new dept, no employees yet

-- Employees (NULL ManagerID = top of the hierarchy)
INSERT INTO Employee VALUES
(101, 'Ali Khan', 'M', 120000, '2018-03-15', 'Lahore', NULL, 10),
(102, 'Sara Iqbal', 'F', 95000, '2019-06-01', 'Lahore', 101, 10),
(103, 'Hamza Raza', 'M', 85000, '2020-01-20', 'Karachi', 101, 10),
(104, 'Ayesha Noor', 'F', 110000, '2017-11-10', 'Karachi', NULL, 20),
(105, 'Bilal Ahmed', 'M', 70000, '2021-04-05', 'Karachi', 104, 20),
(106, 'Fatima Sheikh', 'F', 90000, '2019-09-12', 'Islamabad', NULL, 30),
(107, 'Usman Tariq', 'M', 78000, '2022-02-18', 'Islamabad', 106, 30),
(108, 'Maira Javed', 'F', 115000, '2016-07-22', 'Lahore', NULL, 40),
(109, 'Zain Abbas', 'M', 60000, '2023-01-09', 'Lahore', 108, 40),
(110, 'Nida Yousaf', 'F', 72000, '2022-08-30', NULL, 108, 40);

-- Projects
INSERT INTO Project VALUES
(1001, 'Website Revamp', '2024-01-10', '2024-06-30', 10),
(1002, 'Mobile App', '2024-03-01', '2024-12-31', 10),
(1003, 'Brand Campaign', '2024-02-15', '2024-05-15', 20),
(1004, 'Audit System', '2024-04-01', NULL, 30),
(1005, 'AI Research', '2024-05-01', '2025-04-30', 40),
(1006, 'Internal Tool', '2024-06-01', '2024-09-30', NULL); -- no dept yet

-- Assignments (note: employees 107 and 110 are NOT assigned, project 1006 has no staff)
INSERT INTO Assignment VALUES
(101, 1001, 10),
(102, 1001, 20),
(102, 1002, 15),
(103, 1002, 30),
(104, 1003, 25),
(105, 1003, 40),
(106, 1004, 35),
(108, 1005, 20),
(109, 1005, 30);

```

Important — the schema's deliberate gaps

Department 50 (Sales) has no employees. **Project 1006** has no department. **Employees 107 and 110** have no project assignments. Top-level managers (101, 104, 106, 108) have NULL ManagerID. These edge cases exist so LEFT/RIGHT joins, IS NULL filters, and the anti-join pattern produce meaningful results in your queries.

4. Why Do We Need Joins?

After normalization, related data lives in separate tables. The Employee table stores *DeptID* but not *DeptName*. To answer the question *'show every employee with their department name'*, MySQL must **join** the two tables back together — matching rows where the foreign key in Employee equals the primary key in Department.

4.1 The Cartesian Product (and Why We Avoid It)

If you list two tables in FROM without a join condition, MySQL pairs every row of the first with every row of the second. With 10 employees and 5 departments, that's 50 mostly-meaningless rows.

```
-- Cartesian product: 10 x 5 = 50 rows, mostly nonsense
SELECT e.EmpName, d.DeptName
FROM Employee e, Department d;
```

The fix is a **join condition** — usually equality between a foreign key and a primary key — that restricts the result to rows that genuinely belong together.

5. Join Types at a Glance

Join	Returns
INNER JOIN	Only rows where the join condition matches in both tables.
LEFT JOIN (LEFT OUTER)	All rows from the left table; matching rows from the right (NULL where no match).
RIGHT JOIN (RIGHT OUTER)	All rows from the right table; matching rows from the left (NULL where no match).
FULL OUTER JOIN	All rows from both tables, matched where possible. Not native in MySQL — emulate with UNION.
CROSS JOIN	Cartesian product: every row of A combined with every row of B. Rarely useful directly.
SELF JOIN	A table joined to itself, using table aliases. Used for hierarchies and pairings.

6. INNER JOIN

Returns only rows where the join condition matches on both sides. Any row without a match (e.g. a department with no employees, or an employee with no department) is dropped.

Class Example 1 — Each employee with their department name:

```
SELECT e.EmpID, e.EmpName, d.DeptName, d.Location
FROM Employee e
INNER JOIN Department d ON e.DeptID = d.DeptID;
```

Result: 10 employees with their department name. Department 50 (Sales) is **not** in the output because no one works there.

Class Example 2 — INNER JOIN with a WHERE filter:

```
SELECT e.EmpName, d.DeptName, e.Salary
FROM Employee e
INNER JOIN Department d ON e.DeptID = d.DeptID
WHERE e.Salary > 90000
ORDER BY e.Salary DESC;
```

Aliasing tables

`Employee e` creates the alias `e`. Aliases make queries shorter and are **required** in self-joins. The keyword `AS` is optional: `Employee AS e` works identically. The keyword `INNER` is also optional — plain `JOIN` means `INNER JOIN`.

7. LEFT JOIN

Keeps every row of the **left** table, even those without a match in the right table. Use this when you want to ensure no row from the primary table is dropped.

Class Example 3 — Every employee, with their project (if any):

```
SELECT e.EmpName, p.ProjectName, a.HoursPerWeek
FROM Employee e
LEFT JOIN Assignment a ON e.EmpID = a.EmpID
LEFT JOIN Project p ON a.ProjectID = p.ProjectID;
```

Employees 107 (Usman) and 110 (Nida) appear with NULL ProjectName and NULL HoursPerWeek — they have no assignments. With INNER JOIN they would be missing entirely.

Class Example 4 — Find employees with NO project:

```
SELECT e.EmpID, e.EmpName
FROM Employee e
LEFT JOIN Assignment a ON e.EmpID = a.EmpID
WHERE a.EmpID IS NULL;
```

Anti-join pattern

LEFT JOIN ... WHERE right_key IS NULL is the standard way to find rows in the left table with **no match** in the right. Memorise this idiom — it is one of the most common patterns in real reporting work.

8. RIGHT JOIN

Mirror image of LEFT JOIN — keeps every row of the **right** table. By convention most query authors prefer LEFT JOIN; any RIGHT JOIN can be rewritten as a LEFT JOIN by swapping the table order.

Class Example 5 — Every department, with its employees:

```
SELECT e.EmpName, d.DeptName
FROM Employee e
RIGHT JOIN Department d ON e.DeptID = d.DeptID;
```

Department 50 (Sales) appears with NULL EmpName because no one works there yet.

Class Example 6 — The same query, rewritten as LEFT JOIN:

```
SELECT e.EmpName, d.DeptName
FROM Department d
LEFT JOIN Employee e ON e.DeptID = d.DeptID;
```

Subtle trap — LEFT JOIN + WHERE

If you write `LEFT JOIN B ON ... WHERE B.col = 'X'`, any row where B is NULL (no match) will fail the WHERE check and be dropped — silently turning your LEFT JOIN into an INNER JOIN. To preserve unmatched rows, move the condition into the ON clause: `LEFT JOIN B ON A.x = B.x AND B.col = 'X'`.

9. FULL OUTER JOIN (via UNION)

MySQL does not implement FULL OUTER JOIN directly. Emulate it by combining a LEFT and a RIGHT JOIN with UNION:

```
SELECT e.EmpName, d.DeptName
FROM Employee e
LEFT JOIN Department d ON e.DeptID = d.DeptID

UNION

SELECT e.EmpName, d.DeptName
FROM Employee e
RIGHT JOIN Department d ON e.DeptID = d.DeptID;
```

UNION automatically removes duplicate rows. The result includes every employee (matched or not) and every department (matched or not).

10. SELF JOIN

A table joined to itself. The Employee table has a **ManagerID** column that points to another row in the same table. To list each employee with their manager's name, we treat Employee as two virtual tables using aliases.

Class Example 7 — Each employee with their manager's name:

```
SELECT e.EmpName AS Employee,
       m.EmpName AS Manager
FROM   Employee e
LEFT JOIN Employee m ON e.ManagerID = m.EmpID;
```

Top-level managers (Ali, Ayesha, Fatima, Maira) have NULL ManagerID, so their Manager column is NULL. The LEFT JOIN keeps them in the output.

Class Example 8 — Employees who earn more than their manager:

```
SELECT e.EmpName AS Employee, e.Salary AS EmpSalary,
       m.EmpName AS Manager, m.Salary AS MgrSalary
FROM   Employee e
INNER JOIN Employee m ON e.ManagerID = m.EmpID
WHERE  e.Salary > m.Salary;
```

11. Joining More Than Two Tables

Joins chain naturally. To list every employee, the project they work on, and the project's department, we join four tables:

```
SELECT e.EmpName, p.ProjectName, d.DeptName, a.HoursPerWeek
FROM Employee e
JOIN Assignment a ON e.EmpID      = a.EmpID
JOIN Project    p ON a.ProjectID = p.ProjectID
JOIN Department d ON p.DeptID    = d.DeptID
ORDER BY e.EmpName;
```


Build joins one step at a time

When writing a multi-table query, start with two tables. Run the query and check the row count. Then add the next JOIN and run again. If a row count suddenly explodes, the new join condition is wrong; if it shrinks unexpectedly, you needed LEFT JOIN instead of INNER JOIN.

12. Lab Tasks

Solve each task in MySQL using the Company schema from section 3. Save all queries in a single file named `RollNo_Joins.sql`, with a comment line before each answer (e.g. `-- Task A1`).

Part A — INNER, LEFT, and RIGHT Joins

#	Task
A1	List every employee with their department name and location. (INNER JOIN)
A2	Same as A1, but include employees whose DeptID is NULL — if any. (LEFT JOIN)
A3	List every department with the names of its employees. Departments with no employees should still appear once with NULL EmpName.
A4	List every project with its department name and location. Include projects that have no department.
A5	Find employees who are not assigned to any project. (LEFT JOIN + IS NULL pattern)
A6	List every project that currently has no assignments.
A7	Show every employee in the Engineering department along with their salary, sorted by salary descending. (INNER JOIN + WHERE)
A8	List employees in Lahore-based departments. Show EmpName and DeptName.
A9	List every department and the count of how many employees work there (use LEFT JOIN with COUNT and GROUP BY). Include departments with zero employees.
A10	Produce a FULL OUTER JOIN result of Employee and Department using UNION.

Part B — Self Joins, Multi-table Joins, and Combined Challenges

#	Task
B1	For each employee, show their name and their manager's name. Top-level managers should still appear with NULL Manager. (SELF JOIN)
B2	List employees who earn more than their direct manager. Show employee name, employee salary, manager name, manager salary.
B3	List employees whose manager works in a different department . Show EmpName, ManagerName, and both department names. (Hint: self-join Employee, then join Department twice with different aliases.)
B4	Show every employee with the project name they work on and weekly hours. (3-table join: Employee → Assignment → Project)
B5	List every assignment with employee name, project name, and the project's department name. (4-table join)
B6	List the names and weekly hours of employees working on the Mobile App project.
B7	List every employee in Lahore together with the projects they are assigned to (project name and hours). Include Lahore employees with no assignments.
B8	List the names of employees who work on a project run by a department different from their own . (Compare e.DeptID and p.DeptID.)
B9	For each department, list the names of projects that started in 2024. Include departments that have no such projects. (LEFT JOIN + WHERE on date)
B10	List every employee with the total hours they work per week across all their projects. Include employees with zero hours. (LEFT JOIN + SUM + GROUP BY)

Tip

If a query returns the wrong number of rows: comment out the WHERE clause first and check that the join skeleton (FROM + JOIN + ON) returns the expected count. Most join bugs are in the ON clause, not the WHERE.

13. Assessment Problem (Solve Individually)

Scenario — Library Database. A small public library uses the schema below. Build it in MySQL, load the provided sample data, and answer the ten questions that follow. *This problem is graded.*

13.1 Schema

Table	Columns	Keys
Author	AuthorID, AuthorName, Country	PK: AuthorID
Book	BookID, Title, Genre, Price, AuthorID, PublishedYear	PK: BookID; FK: AuthorID → Author
Member	MemberID, MemberName, City, JoinDate	PK: MemberID
Loan	LoanID, MemberID, BookID, LoanDate, ReturnDate	PK: LoanID; FK: MemberID; FK: BookID

13.2 Setup Script

```
CREATE DATABASE IF NOT EXISTS library_lab;
USE library_lab;

DROP TABLE IF EXISTS Loan, Book, Member, Author;

CREATE TABLE Author (
    AuthorID    INT PRIMARY KEY,
    AuthorName  VARCHAR(60) NOT NULL,
    Country     VARCHAR(30)
);

CREATE TABLE Book (
    BookID      INT PRIMARY KEY,
    Title       VARCHAR(80) NOT NULL,
    Genre       VARCHAR(30),
    Price       DECIMAL(8,2),
    AuthorID    INT,
    PublishedYear INT,
    FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID)
);

CREATE TABLE Member (
    MemberID    INT PRIMARY KEY,
    MemberName  VARCHAR(60) NOT NULL,
    City        VARCHAR(30),
    JoinDate    DATE
);

CREATE TABLE Loan (
    LoanID      INT PRIMARY KEY,
    MemberID    INT,
    BookID      INT,
    LoanDate    DATE,
```

```
ReturnDate DATE,          -- NULL = not yet returned
FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
FOREIGN KEY (BookID) REFERENCES Book(BookID)
);

INSERT INTO Author VALUES
(1, 'Jane Austen',      'UK'),
(2, 'Chinua Achebe',    'Nigeria'),
(3, 'Haruki Murakami',  'Japan'),
(4, 'Bapsi Sidhwa',     'Pakistan'),
(5, 'Mohsin Hamid',     'Pakistan'),
(6, 'Anonymous Writer', NULL); -- no books

INSERT INTO Book VALUES
(101, 'Pride and Prejudice', 'Fiction', 850.00, 1, 1813),
(102, 'Emma',                'Fiction', 900.00, 1, 1815),
(103, 'Things Fall Apart',   'Fiction', 1100.00, 2, 1958),
(104, 'Norwegian Wood',      'Fiction', 1500.00, 3, 1987),
(105, 'Kafka on the Shore',  'Fiction', 1700.00, 3, 2002),
(106, 'Ice-Candy-Man',       'Fiction', 1200.00, 4, 1988),
(107, 'The Reluctant Fundamentalist', 'Fiction', 1300.00, 5, 2007),
(108, 'Exit West',           'Fiction', 1450.00, 5, 2017),
(109, 'Mystery Title',       'Mystery', 950.00, NULL, 2020); -- no author

INSERT INTO Member VALUES
(201, 'Ahmad Raza', 'Lahore', '2023-01-15'),
(202, 'Sara Imran', 'Karachi', '2023-03-20'),
(203, 'Bilal Khan', 'Lahore', '2024-02-10'),
(204, 'Fatima Ali', 'Islamabad', '2022-09-05'),
(205, 'Hira Yousaf', NULL, '2024-05-01'); -- no loans yet

INSERT INTO Loan VALUES
(1, 201, 101, '2024-03-01', '2024-03-15'),
(2, 201, 104, '2024-04-10', NULL),
(3, 202, 103, '2024-02-20', '2024-03-05'),
(4, 202, 107, '2024-05-01', NULL),
(5, 203, 105, '2024-04-25', '2024-05-15'),
(6, 204, 102, '2024-01-10', '2024-01-30'),
(7, 204, 108, '2024-06-01', NULL);
```

13.3 Questions

#	Question	Marks
Q1	Show every book with its author's name and country. (INNER JOIN)	10
Q2	Show every author with their books. Authors with no books must still appear once with NULL Title. (LEFT JOIN)	10
Q3	List members who have never borrowed any book . (LEFT JOIN + IS NULL)	10
Q4	List every loan with the member's name, book title, and author's name. (3-table join)	15
Q5	List currently borrowed books (ReturnDate IS NULL) along with the borrower's name and city.	10
Q6	List Pakistani authors and the titles of their books. Include Pakistani authors with no books. (LEFT JOIN + WHERE on author country)	15
Q7	List every book together with the names of all members who have borrowed it. Include books that have never been borrowed. (LEFT JOIN chain)	10
Q8	Find authors whose books have never been borrowed. (Multi-step: Author → Book → Loan)	10
Q9	Produce a FULL OUTER JOIN of Author and Book using UNION — every author and every book, matched where possible.	5
Q10	List members who have borrowed books written by Pakistani authors. Show member name, book title, and author name. (4-way join with filter)	5

13.4 Submission

- Push your RollNo_Joins.sql file to your existing GitHub repository in a folder named Lab2-Joins/.
- The file must contain: the setup script (or a comment confirming you ran it), all Lab Task answers (Parts A and B), and all 10 Assessment answers, each labelled with a comment (e.g. -- Q5).
- Share the GitHub link to that folder before the deadline announced in class.

14. Common Mistakes to Avoid

- Forgetting the **ON** clause — MySQL produces a Cartesian product (huge nonsense result).
- Using INNER JOIN when you needed LEFT JOIN, then wondering why some rows 'disappeared'.
- Putting a right-table condition in WHERE on a LEFT JOIN — silently turns it into an INNER JOIN. Move the condition into the ON clause.
- Forgetting table aliases in self-joins. Without aliases MySQL cannot tell which copy of the table you mean.
- Joining on the wrong columns. Always join FK to PK; mismatched data types (e.g. INT to VARCHAR) silently produce zero matches.
- Building a 4-table join without testing the 2- and 3-table versions first. Bugs become very hard to find.
- Using `SELECT *` in a multi-table join — you get duplicate columns (e.g. DeptID twice). Always list specific qualified columns.
- Forgetting that JOIN means INNER JOIN by default. Be explicit when teaching others or reading code.

15. Appendix — Quick Reference

```
-- INNER JOIN (only matching rows)
SELECT a.*, b.col
FROM A a
INNER JOIN B b ON a.x = b.x;

-- LEFT JOIN (all of A, NULLs where no B match)
SELECT a.*, b.col
FROM A a
LEFT JOIN B b ON a.x = b.x;

-- RIGHT JOIN (all of B, NULLs where no A match)
SELECT a.col, b.*
FROM A a
RIGHT JOIN B b ON a.x = b.x;

-- FULL OUTER JOIN (MySQL workaround)
SELECT a.col, b.col FROM A a LEFT JOIN B b ON a.x = b.x
UNION
SELECT a.col, b.col FROM A a RIGHT JOIN B b ON a.x = b.x;

-- SELF JOIN (each employee + manager)
SELECT e.EmpName AS Employee, m.EmpName AS Manager
FROM Employee e
LEFT JOIN Employee m ON e.ManagerID = m.EmpID;

-- ANTI-JOIN (rows in A with NO match in B)
SELECT A.*
FROM A
LEFT JOIN B ON A.x = B.x
WHERE B.x IS NULL;

-- MULTI-TABLE JOIN (chain joins one at a time)
SELECT e.EmpName, p.ProjectName, d.DeptName
FROM Employee e
```

```
JOIN Assignment a ON e.EmpID      = a.EmpID
JOIN Project    p ON a.ProjectID = p.ProjectID
JOIN Department d ON p.DeptID     = d.DeptID;

-- COMBINED WITH FILTERS FROM LAB #1
SELECT e.EmpName, d.DeptName
FROM Employee e
INNER JOIN Department d ON e.DeptID = d.DeptID
WHERE d.Location = 'Lahore'
AND e.Salary > 90000
ORDER BY e.Salary DESC;
```

End of Lab Manual #2 — SQL Joins.